

ValorantSense

Relaciones entre tablas
Explicacion de todas las interrelaciones del diagrama E/R

Proyecto	ValorantSense — Plataforma web para jugadores de Valorant
Contenido	Interrelaciones del diagrama: cardinalidad, tablas implicadas y funcionamiento
Basado en	Diagrama Entidad/Relacion de ValorantSense y su base de datos SQL

Indice de contenidos

Indice de contenidos	2
1. Introduccion	3
1.1 Resumen de todas las interrelaciones	3
2. Detalle de cada interrelacion	4
2.1 tiene_meta — AGENTE y AGENTE_MAPA_META	4
2.2 tiene_favorito — AGENTE y USUARIO	4
2.3 aparece_en — AGENTE y LINEUP	4
2.4 sube — USUARIO y LINEUP	5
2.5 genera — USUARIO y RUTINA	5
2.6 publica — USUARIO y SOLICITUD_EQUIPO	5
2.7 envia_mensaje — USUARIO con USUARIO (reflexiva)	6
2.8 tiene_amistad — USUARIO con USUARIO (reflexiva)	6
3. Tablas sin interrelaciones	7
3.1 ENTRENAMIENTO_OPCION	7
3.2 ENTRENAMIENTO_VIDEO	7
4. Tipos de correspondencia usados	8
4.1 Correspondencia 1 : N	8
4.2 Correspondencia N : M	8
4.3 Relaciones reflexivas	8

1. Introduccion

Este documento explica todas las interrelaciones que aparecen en el diagrama Entidad/Relacion de ValorantSense. Para cada relacion se indica que tablas conecta, el tipo de correspondencia (1:N o N:M), la cardinalidad exacta de cada extremo, como se implementa en la base de datos y que significa en el funcionamiento real de la plataforma.

El diagrama tiene ocho interrelaciones en total. Seis son relaciones entre entidades distintas y dos son relaciones reflexivas, es decir, una entidad que se relaciona consigo misma. Ademas hay dos tablas de contenido que no tienen ninguna clave foranea y por tanto no participan en ninguna interrelacion.

1.1 Resumen de todas las interrelaciones

La siguiente tabla recoge de un vistazo todas las relaciones del diagrama:

Interrelacion	Entidad A	Card. A	Card. B	Entidad B	Tipo
tiene_meta	AGENTE	(1,n)	(1,1)	AGENTE_MAPA_META	1:N
tiene_favorito	AGENTE	(0,n)	(0,n)	USUARIO	N:M
aparece_en	AGENTE	(1,n)	(1,1)	LINEUP	1:N
sube	USUARIO	(1,n)	(0,1)	LINEUP	1:N
genera	USUARIO	(1,n)	(1,1)	RUTINA	1:N
publica	USUARIO	(1,n)	(1,1)	SOLICITUD_EQUIPO	1:N
envia_mensaje	USUARIO	(0,n)	(0,n)	USUARIO	N:M reflexiva
tiene_amistad	USUARIO	(0,n)	(0,n)	USUARIO	N:M reflexiva

2. Detalle de cada interrelacion

2.1 tiene_meta — AGENTE y AGENTE_MAPA_META

Tablas que conecta	Tipo	Cardinalidad
AGENTE → AGENTE_MAPA_META	1 : N	AGENTE (1,n) — AGENTE_MAPA_META (1,1)

Implementacion en SQL: La tabla AGENTE_MAPA_META contiene la columna agente_id como clave foranea que referencia a AGENTE(id). Por cada agente hay multiples registros de meta, uno por cada mapa del juego.

Un agente puede tener valoracion de meta en varios mapas, pero cada registro de meta pertenece a exactamente un agente. Esto significa que se pueden tener datos como 'Jett es tier S en Ascent y tier A en Bind'. La cardinalidad (1,n) en AGENTE indica que todo agente tiene meta en al menos un mapa. La cardinalidad (1,1) en AGENTE_MAPA_META indica que cada registro de meta pertenece siempre a un unico agente. En la aplicacion, el selector de composicion de equipo consulta esta tabla para sugerir los mejores agentes segun el mapa elegido por el usuario.

2.2 tiene_favorito — AGENTE y USUARIO

Tablas que conecta	Tipo	Cardinalidad
AGENTE ↔ USUARIO (via AGENTE_FAVORITO)	N : M	AGENTE (0,n) — USUARIO (0,n)

Implementacion en SQL: Se implementa mediante la tabla puente AGENTE_FAVORITO que contiene las columnas usuario_id (FK a USUARIO) y agente_id (FK a AGENTE). Esta tabla representa cada par usuario-agente favorito.

Un usuario puede marcar varios agentes como favoritos y un mismo agente puede ser favorito de muchos usuarios a la vez. La cardinalidad (0,n) en ambos lados indica que un usuario puede no tener ningun favorito marcado y que un agente puede no ser favorito de nadie. Los agentes favoritos se muestran en el perfil publico del usuario y tambien aparecen en las tarjetas del matchmaker para que otros jugadores vean con que agentes suele jugar cada persona antes de enviarle una solicitud de equipo.

2.3 aparece_en — AGENTE y LINEUP

Tablas que conecta	Tipo	Cardinalidad
AGENTE → LINEUP	1 : N	AGENTE (1,n) — LINEUP (1,1)

Implementacion en SQL: La tabla LINEUP contiene la columna agente_id como clave foranea que referencia a AGENTE(id). Cada lineup tiene exactamente un agente asociado.

Un agente puede aparecer en muchos lineups publicados por distintos usuarios, pero cada lineup pertenece a un unico agente. La cardinalidad (1,n) en AGENTE indica que existe al menos un lineup por agente en la base de datos. La cardinalidad (1,1) en LINEUP indica que todo lineup debe especificar obligatoriamente para que agente sirve la tecnica. En la aplicacion, el filtro de la biblioteca de lineups usa esta relacion para mostrar solo los lineups del agente seleccionado en el desplegable.

2.4 sube — USUARIO y LINEUP

Tablas que conecta	Tipo	Cardinalidad
USUARIO → LINEUP	1 : N	USUARIO (1,n) — LINEUP (0,1)

Implementacion en SQL: La tabla LINEUP contiene la columna usuario_id como clave foranea que referencia a USUARIO(id). Esta columna admite valores nulos para el caso en que la cuenta del autor se haya eliminado.

Un usuario puede subir muchos lineups a lo largo del tiempo, pero cada lineup ha sido publicado por como maximo un usuario. La cardinalidad (1,n) en USUARIO indica que existe al menos un lineup subido por alguno de los usuarios del sistema. La cardinalidad (0,1) en LINEUP indica que un lineup puede no tener usuario asociado si la cuenta del autor se elimino despues de publicarlo. En la aplicacion, al acceder al perfil de un usuario se pueden ver todos los lineups que ha subido consultando esta relacion. Un administrador puede aprobar o rechazar lineups filtrando por usuario_id.

2.5 genera — USUARIO y RUTINA

Tablas que conecta	Tipo	Cardinalidad
USUARIO → RUTINA	1 : N	USUARIO (1,n) — RUTINA (1,1)

Implementacion en SQL: La tabla RUTINA contiene la columna usuario_id como clave foranea que referencia a USUARIO(id). Cada rutina guardada pertenece a un usuario concreto.

Un usuario puede tener guardadas muchas rutinas de entrenamiento generadas en diferentes momentos, pero cada rutina pertenece exactamente a un usuario. La cardinalidad (1,n) en USUARIO indica que algun usuario ha generado al menos una rutina. La cardinalidad (1,1) en RUTINA indica que toda rutina guardada debe pertenecer a un usuario existente. En la aplicacion, cuando el usuario genera una rutina personalizada eligiendo su rango y las categorias que quiere trabajar, el resultado se guarda en esta tabla asociado a su cuenta. Puede volver a consultarla desde su historial sin tener que generarla de nuevo.

2.6 publica — USUARIO y SOLICITUD_EQUIPO

Tablas que conecta	Tipo	Cardinalidad
USUARIO → SOLICITUD_EQUIPO	1 : N	USUARIO (1,n) — SOLICITUD_EQUIPO (1,1)

Implementacion en SQL: La tabla SOLICITUD_EQUIPO contiene la columna usuario_id como clave foranea que referencia a USUARIO(id). Cada solicitud ha sido publicada por un usuario.

Un usuario puede publicar varias solicitudes de busqueda de equipo a lo largo del tiempo, pero cada solicitud pertenece a un unico usuario. La cardinalidad (1,n) en USUARIO indica que existe al menos una solicitud publicada en el sistema. La cardinalidad (1,1) en SOLICITUD_EQUIPO indica que toda solicitud debe tener un usuario que la haya creado. En la aplicacion, el matchmaker muestra las solicitudes activas de otros usuarios compatibles en rango con el jugador que esta buscando equipo. Al desactivar una solicitud el campo activa cambia a falso pero el registro se conserva.

2.7 envia_mensaje — USUARIO con USUARIO (reflexiva)

Tablas que conecta	Tipo	Cardinalidad	Atributos propios
USUARIO ↔ USUARIO (via MENSAJE)	N : M reflexiva	USUARIO emisor (0,n) — USUARIO receptor (0,n)	contenido, tipo, leído, creado_en

Implementacion en SQL: Se implementa mediante la tabla MENSAJE que contiene dos claves foraneas hacia USUARIO: emisor_id (quien envia) y receptor_id (quien recibe). Ambas referencian a la misma tabla USUARIO.

Un usuario puede enviar mensajes a muchos otros usuarios y puede recibir mensajes de muchos otros. Es una relacion reflexiva porque los dos extremos apuntan a la misma entidad USUARIO. La cardinalidad (0,n) en ambos lados indica que un usuario puede no haber enviado ni recibido ningun mensaje todavia. La tabla MENSAJE tiene ademas atributos propios de la relacion: el contenido del mensaje, el tipo (texto normal, codigo de Valorant, enlace de Discord o Riot ID), si ya ha sido leído y la fecha de envio. En la aplicacion, solo pueden intercambiar mensajes los usuarios que tienen una amistad aceptada, lo que se comprueba en el controlador antes de guardar el mensaje.

2.8 tiene_amistad — USUARIO con USUARIO (reflexiva)

Tablas que conecta	Tipo	Cardinalidad	Atributos propios
USUARIO ↔ USUARIO (via AMISTAD)	N : M reflexiva	USUARIO emisor (0,n) — USUARIO receptor (0,n)	estado, creado_en, resuelto_en

Implementacion en SQL: Se implementa mediante la tabla AMISTAD que contiene dos claves foraneas hacia USUARIO: emisor_id (quien envia la solicitud) y receptor_id (quien la recibe). Ambas referencian a la misma tabla USUARIO.

Un usuario puede tener amistad con muchos otros y ser amigo de muchos. Es una relacion reflexiva porque los dos extremos apuntan a la misma entidad USUARIO. La cardinalidad (0,n) en ambos lados indica que un usuario puede no tener ninguna amistad todavia. La tabla AMISTAD tiene atributos propios de la relacion: el estado actual (pendiente, aceptada o rechazada), la fecha en que se envio la solicitud y la fecha en que se resolvió si ya no esta pendiente. En la aplicacion, al enviar una solicitud de amistad el estado es pendiente. El receptor puede aceptarla, lo que cambia el estado a aceptada y permite el chat entre ambos, o rechazarla. Desde ajustes se puede eliminar una amistad borrando el registro correspondiente.

3. Tablas sin interrelaciones

Las siguientes dos tablas no participan en ninguna interrelacion del diagrama porque no tienen claves foraneas. Son tablas de contenido independientes que el sistema consulta directamente segun parametros de busqueda sin depender de otras tablas.

3.1 ENTRENAMIENTO_OPCION

Almacena ejercicios de entrenamiento clasificados por rango y categoria. No tiene ninguna clave foranea hacia otra tabla. El sistema la consulta filtrando por rango (del usuario actual) y por categoria (las que el usuario selecciona en los checkboxes) para elegir que ejercicios incluir en la rutina generada. Los resultados se copian al texto de la nueva RUTINA que si queda vinculada al usuario.

3.2 ENTRENAMIENTO_VIDEO

Guarda un video de referencia por cada combinacion unica de rango y categoria. No tiene ninguna clave foranea. El sistema la consulta directamente con los mismos parametros (rango del usuario y categoria elegida) para mostrar el video de formacion correspondiente. La restriccion de unicidad sobre rango y categoria garantiza que hay exactamente un video de referencia por combinacion posible.

Tabla	Claves foraneas	Como se usa sin relacion
ENTRENAMIENTO_OPCION	Ninguna	Se filtra por rango y categoria para seleccionar ejercicios de la rutina
ENTRENAMIENTO_VIDEO	Ninguna	Se filtra por rango y categoria para mostrar el video de referencia al usuario

4. Tipos de correspondencia usados

En el diagrama E/R de ValorantSense aparecen dos tipos de correspondencia: 1:N y N:M. A continuacion se explica que significa cada uno en el contexto de este proyecto.

4.1 Correspondencia 1 : N

Una ocurrencia de la entidad A puede relacionarse con muchas ocurrencias de B, pero cada ocurrencia de B solo puede relacionarse con una de A. En la base de datos se implementa anadiendo la clave primaria de A como clave foranea en la tabla B.

Ejemplos en ValorantSense: un AGENTE puede tener meta en muchos mapas (tiene_meta), un USUARIO puede subir muchos lineups (sube) y puede generar muchas rutinas (genera). En todos estos casos la clave foranea va en la tabla del lado N.

4.2 Correspondencia N : M

Una ocurrencia de A puede relacionarse con muchas de B y viceversa. No se puede implementar directamente con una clave foranea, sino que requiere una tabla intermedia que contiene las claves primarias de ambas entidades.

Ejemplos en ValorantSense: AGENTE y USUARIO (tiene_favorito) se implementa mediante la tabla AGENTE_FAVORITO. Las dos relaciones reflexivas de USUARIO consigo mismo (envia_mensaje y tiene_amistad) se implementan mediante las tablas MENSAJE y AMISTAD respectivamente, que contienen dos claves foraneas hacia USUARIO.

4.3 Relaciones reflexivas

Una relacion reflexiva ocurre cuando una entidad se relaciona consigo misma. En ValorantSense hay dos: envia_mensaje y tiene_amistad, ambas sobre USUARIO. En el diagrama se representa con el rombo conectado dos veces a la misma entidad. En la base de datos se implementa igual que un N:M: con una tabla intermedia que tiene dos claves foraneas apuntando a la misma tabla, una para el rol de emisor y otra para el rol de receptor.